MM811 Programming Assignment Report, 2026

Nathan Thai (nathai@ualberta.ca)
All code for submitted sections can be found at
https://github.com/NathanThai2201/MM811_Assignment.git

1. IMAGE CLASSIFICATION ARCHITECTURE

For this section will use the MNIST dataset, we will use the entire dataset for training. For the model architecture, this is very similar to LeNet5, albeit with less channels. We keep the spatial dimensions the same.

Table 1: Details of ModifiedLeNet architecture.

Type	weight	stride	padding	Data size
Layer 1				
Input				$N \times 1 \times 28 \times 28$
Conv2d	$5 \times 1 \times 5 \times 5$	1	2	$N \times 5 \times 28 \times 28$
BatchNorm2d	5			$N \times 5 \times 28 \times 28$
Sigmoid				$N \times 5 \times 28 \times 28$
MaxPool2d		2	0	$N \times 5 \times 14 \times 14$
Conv2d	$10 \times 5 \times 5 \times 5$	1	0	$N \times 10 \times 10 \times 10$
BatchNorm2d	10			$N \times 10 \times 10 \times 10$
Sigmoid				$N \times 10 \times 10 \times 10$
MaxPool2d		2	0	$N \times 10 \times 5 \times 5$
Fully Connected				
Flatten				$N \times 250$
Linear	120×250			$N \times 120$
Sigmoid				$N \times 120$
Linear	84×120			$N \times 84$
Sigmoid				$N \times 84$
Linear (Head)	num_classes $\times$ 84			$N \times$ num_classes

The model's outputs reach approximately 98% accuracy.

```
Accuracy of the network on the 10000 test images: 98.48 %
```

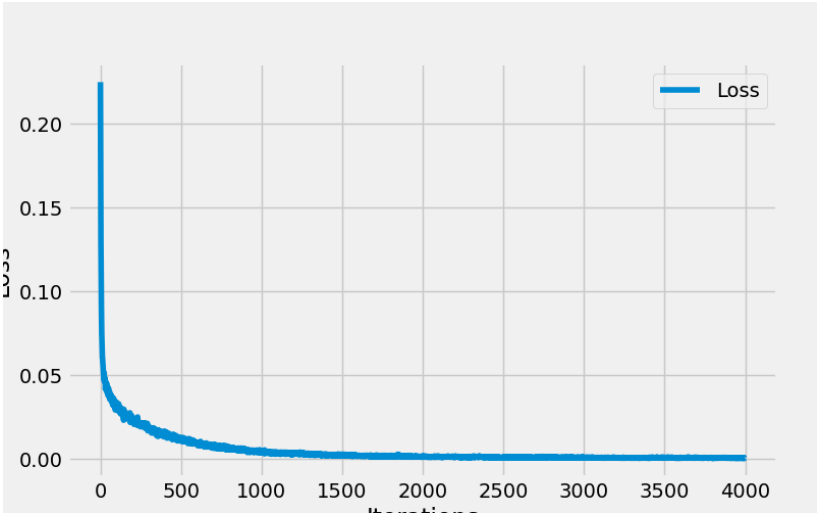
2. CAPTURING BINARY LATENT REPRESENTATIONS

For this section we continue to use the MNIST dataset, we take a 256 image subset for training and memorization. We use the following model architecture:

Table 2: Details of Autoencoder architecture.

Type	weight	stride	padding	Data size
Encoder				
Input				$N \times 1 \times 28 \times 28$
Conv2d	$128 \times 1 \times 5 \times 5$	1	0	$N \times 128 \times 24 \times 24$
BatchNorm2d	128			$N \times 128 \times 24 \times 24$
ReLU				$N \times 128 \times 24 \times 24$
MaxPool2d		2	0	$N \times 128 \times 12 \times 12$
Conv2d	$128 \times 128 \times 3 \times 3$	1	1	$N \times 128 \times 12 \times 12$
BatchNorm2d	128			$N \times 128 \times 12 \times 12$
ReLU				$N \times 128 \times 12 \times 12$
Conv2d	$3 \times 128 \times 5 \times 5$	1	0	$N \times 3 \times 8 \times 8$
MaxPool2d		2	0	$N \times 3 \times 4 \times 4$
BatchNorm2d	3			$N \times 3 \times 4 \times 4$
Latents				
Tanh / STE	Binary ($\{-1, 1\}$)			$N \times 3 \times 4 \times 4$
Decoder				
ConvTranspose2d	$3 \times 128 \times 8 \times 8$	3	2	$N \times 128 \times 13 \times 13$
BatchNorm2d	128			$N \times 128 \times 13 \times 13$
ReLU				$N \times 128 \times 13 \times 13$
Conv2d	$128 \times 128 \times 3 \times 3$	1	1	$N \times 128 \times 13 \times 13$
BatchNorm2d	128			$N \times 128 \times 13 \times 13$
ReLU				$N \times 128 \times 13 \times 13$
ConvTranspose2d	$128 \times 32 \times 4 \times 4$	2	0	$N \times 32 \times 28 \times 28$
BatchNorm2d	32			$N \times 32 \times 28 \times 28$
ReLU				$N \times 32 \times 28 \times 28$
Conv2d	$1 \times 32 \times 3 \times 3$	1	1	$N \times 1 \times 28 \times 28$
Sigmoid				$N \times 1 \times 28 \times 28$

The loss values reaches very low, less than 0.0004 for 1000 epochs.

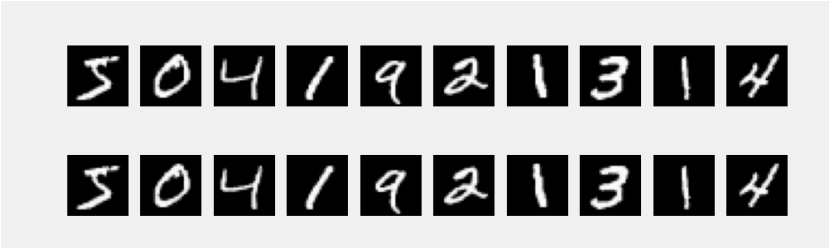


Calculated MSELoss v. Iterations

The model's reconstructed images have very high average PSNR values.

```
Final Average Test PSNR: 35.30 dB
```

Consequently, we also have flawless reconstructions of the training images.



Top: Input images Bottom: Reconstructed images

3. FROM AUTOREGRESSIVE MODEL TO MEMORIZATION MODEL

3.1 SEQUENCE TO TOKEN

A note on implementation, this type of autoregressive model has a limitation when it comes to mapping, - that being one to many mappings. The model predicts an specific output for any given input so it cannot differentiate between two inputs that have large similarities at the beginning.

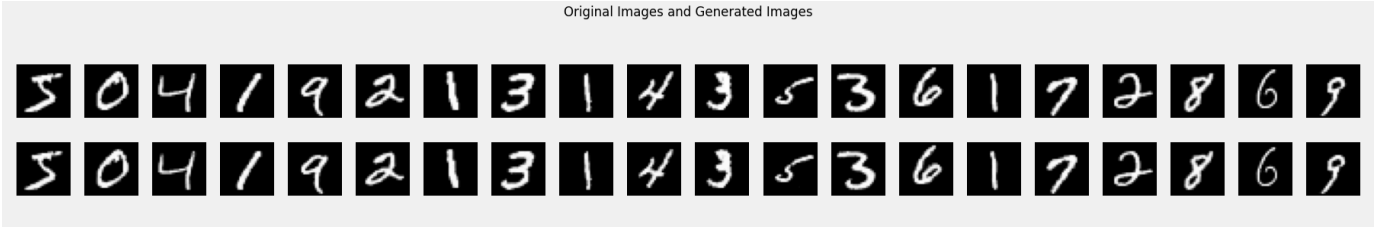
To solve this issue, we add an identifier to each image, in the case of 100 images, there will be an ID between 0 and 99. Since 0 -> 99 represents 100 new token types, we shift the existing decimal tokens by 100, and thus expand the vocab size to 108.

For simplicity across all of our transformers, we use masking to dictate which tokens count toward learning and predicting.

Table 3: Details of SequenceToTokenTransformer architecture.

Type	Configuration	Heads	Layers	Data size
Input & Embeddings				
Token Input	Sequence of token indices			$B \times \text{seq_len}$
Token Embedding	$\text{vocab_size} \rightarrow \text{embed_dim}$			$B \times \text{seq_len} \times 128$
Positional Emb.	Learned parameter (max_len)			$B \times \text{seq_len} \times 128$
Transformer Encoder				
Encoder Layer	Pre-norm, $d_{\text{ff}} = 256$	8	4	$B \times \text{seq_len} \times 128$
Output Head				
Linear Head	$\text{embed_dim} \rightarrow \text{vocab_size}$			$B \times \text{seq_len} \times \text{vocab}$

We have perfect 100% sequence matches of the generated samples with the training samples.



3.2 SEQUENCE TO SEQUENCE

Table 4: Details of Seq2SeqTransformer architecture.

Type	Configuration	Heads	Layers	Data size
Source Stream (Encoder)				
Source Input	Sequence of source token indices			$B \times \text{src_len}$
Source Embedding	$\text{vocab_size} \to \text{embed_dim}$			$B \times \text{src_len} \times 128$
Source Pos. Emb.	Learned parameter (max_len)			$B \times \text{src_len} \times 128$
Encoder Layer	Pre-norm, $d_{\text{ff}} = 256$	8	4	$B \times \text{src_len} \times 128$
Target Stream (Decoder)				
Target Input	Sequence of target token indices			$B \times \text{tgt_len}$
Target Embedding	$(\text{vocab_size} + 1) \to \text{embed_dim}$			$B \times \text{tgt_len} \times 128$
Target Pos. Emb.	Learned parameter (max_len)			$B \times \text{tgt_len} \times 128$
Decoder Layer	Pre-norm, $d_{\text{ff}} = 256$, causal/cross-attn	8	4	$B \times \text{tgt_len} \times 128$
Output Head				
Linear Head	$\text{embed_dim} \to \text{vocab_size}$			$B \times \text{tgt_len} \times \text{vocab_size}$

3.3 SEQUENCE TO HISTOGRAM

For this section we expand the training set to 600 images.

The implementation for this model is very similar to the SequenceToToken model, the only difference being that we return the probabilities of the next token instead of the token itself.

The only difference is this line that returns the probabilities instead of the token.

```
h = self.transformer(emb, mask=causal_mask)
logits = self.head(h)

if return_probs:
  return F.softmax(logits, dim=-1)

return logits
```

Due to the increase in training images, our accuracy of sequence matches is roughly 98%.

4. IMAGE GENERATION

4.1 HIGH-PERFORMANCE AUTOENCODER

For our autoencoders for the CIFAR-10 dataset, we use a ResNet like architecture, with many residual blocks.

Continuous Autoencoder for CIFAR-10:

Table 5: Details of ContinuousAE architecture.

Type	weight / Shape	stride	padding	Data size
Encoder				
Input				$N \times 3 \times 32 \times 32$
Conv2d	$64 \times 3 \times 3 \times 3$	1	1	$N \times 64 \times 32 \times 32$
ResBlock (64)	$2 \times (64 \times 64 \times 3 \times 3)$	1	1	$N \times 64 \times 32 \times 32$
Conv2d	$128 \times 64 \times 4 \times 4$	2	1	$N \times 128 \times 16 \times 16$
ResBlock (128)	$2 \times (128 \times 128 \times 3 \times 3)$	1	1	$N \times 128 \times 16 \times 16$
Conv2d	$256 \times 128 \times 4 \times 4$	2	1	$N \times 256 \times 8 \times 8$
ResBlock (256) $\times 2$	$4 \times (256 \times 256 \times 3 \times 3)$	1	1	$N \times 256 \times 8 \times 8$
GroupNorm + SiLU	8 groups, 256 ch.			$N \times 256 \times 8 \times 8$
Conv2d	latent_channels $\times 256 \times 1 \times 1$	1	0	$N \times$ latent_channels $\times 8 \times 8$
Latents				
Continuous Latent	Float tensor ($C =$ cont_channels)			$N \times C \times 8 \times 8$
Decoder				
Conv2d	$256 \times$ latent_channels $\times 3 \times 3$	1	1	$N \times 256 \times 8 \times 8$
ResBlock (256) $\times 2$	$4 \times (256 \times 256 \times 3 \times 3)$	1	1	$N \times 256 \times 8 \times 8$
ConvTranspose2d	$256 \times 128 \times 4 \times 4$	2	1	$N \times 128 \times 16 \times 16$
ResBlock (128)	$2 \times (128 \times 128 \times 3 \times 3)$	1	1	$N \times 128 \times 16 \times 16$
ConvTranspose2d	$128 \times 64 \times 4 \times 4$	2	1	$N \times 64 \times 32 \times 32$
ResBlock (64)	$2 \times (64 \times 64 \times 3 \times 3)$	1	1	$N \times 64 \times 32 \times 32$
GroupNorm + SiLU	8 groups, 64 ch.			$N \times 64 \times 32 \times 32$
Conv2d	$3 \times 64 \times 3 \times 3$	1	1	$N \times 3 \times 32 \times 32$
Sigmoid				$N \times 3 \times 32 \times 32$

Discrete Autoencoder for CIFAR-10:

Table 6: Details of DiscreteAE architecture modifications and quantization.

Type	weight / Shape	Data size
Backbone Encoder & Decoder		
Encoder / Decoder	Same as ContinuousAE	$N \times$ disc_channels $\times 8 \times 8$
Quantization Layer		
BatchNorm2d	disc_channels (Optional)	$N \times$ disc_channels $\times 8 \times 8$
Discretization	Binary ($\{-1, 1\}$) or Multi-level (K)	$N \times$ disc_channels $\times 8 \times 8$
STE (Straight-Through)	Training pass-through	$N \times$ disc_channels $\times 8 \times 8$

Average PSNR values for the Discrete and Continuous Autoencoders range around 26 to 28.